Enterprise Fullstack Engineering Assessment

Take-home / homework | AI-assisted & vibe coding explicitly allowed

Assessment philosophy
Assessment ini tidak menguji kemampuan coding tanpa AI. Anda boleh menggunakan AI untuk planning, architecture, code generation, debugging, testing, refactoring, dan review. Yang dinilai adalah engineering ownership: apakah solusi end-to-end Anda benar, aman, dapat dijalankan, dapat dijelaskan, dan memiliki evidence bahwa Anda benar-benar mereview hasil tools yang digunakan.

1. Target dan Format

Area	Ketentuan
Role	Fullstack Developer
Format	Take-home/homework. Tidak ada live coding atau live change sebagai bagian wajib assessment.
Target waktu	4-8 jam effective work untuk Phase 1 + Phase 2 + dokumentasi.
Scope	Sengaja kecil dan end-to-end. Prioritaskan correctness dan engineering quality dibanding banyak fitur.
Database	Relational database wajib. SQLite/PostgreSQL/SQL Server/MySQL atau equivalent diperbolehkan.
AI usage	Diperbolehkan penuh, termasuk coding agent yang mengubah banyak file.
Submission	Repository Git dengan history + source + test + migration/schema + dokumentasi.

Timebox rule
Jangan memperpanjang assessment hanya untuk mengejar semua fitur. Jika sudah mendekati 8 jam, berhenti pada state yang stabil, tuliskan apa yang belum selesai di REVIEW.md, dan jelaskan prioritas yang Anda pilih. Kemampuan memilih scope adalah bagian dari penilaian.

2. Business Case yang Harus Dikerjakan

2.1 Latar belakang

Perusahaan masih mengelola permintaan akses aplikasi internal melalui email/chat. Proses approval sulit dilacak, request bisa terkirim dua kali, dan dua approver dapat melakukan action terhadap request yang sama pada waktu berdekatan. Anda diminta membuat MVP **Access Request Hub** sebagai single source of truth untuk request, approval, dan audit trail. Aplikasi cukup berjalan lokal dengan seeded users; integrasi SSO, provisioning, email, atau cloud deployment tidak diperlukan.

2.2 Demo users dan role

Demo user	Role / responsibility
alice@example.local	Requester. Reports to Bob.
bob@example.local	Manager untuk Alice. Memproses Manager approval.
carol@example.local	System Owner untuk aplikasi CRM.

Demo user	Role / responsibility
dana@example.local	System Owner untuk aplikasi Finance Portal.
erin@example.local	Admin/Auditor. Boleh melihat semua request dan audit trail, tetapi bukan approver otomatis.

Authentication boleh disimulasikan dengan user switcher / request header / session lokal. **Authorization tetap wajib ditegakkan di backend.**

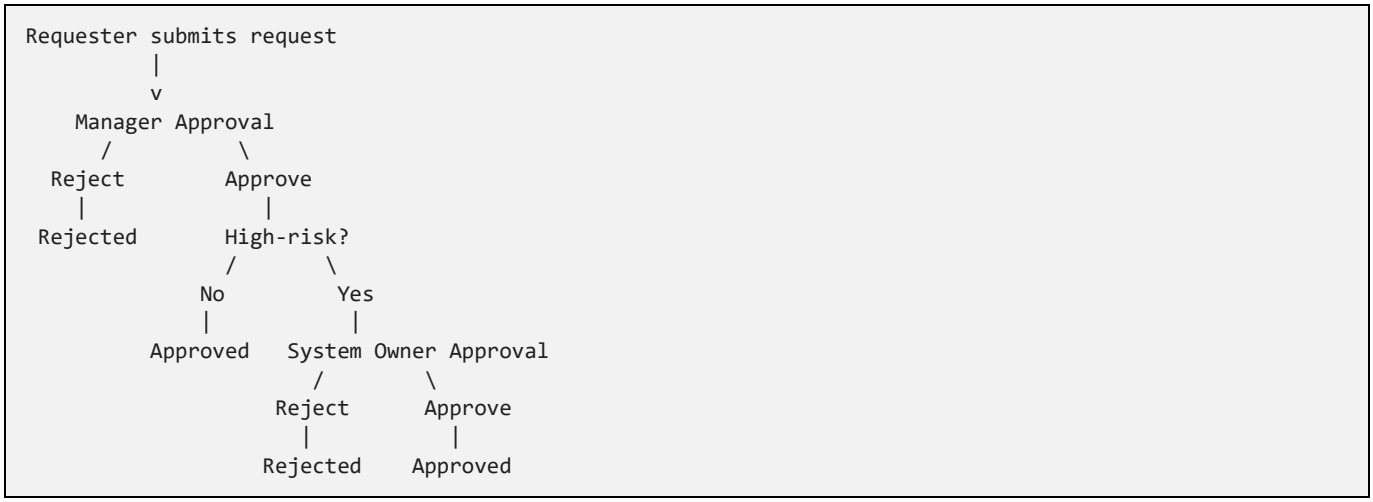
2.3 Master data minimum

- Applications: CRM (System Owner: Carol) dan Finance Portal (System Owner: Dana).
- Environment: NonProduction dan Production.
- Access level: Read dan Admin.
- PolicyVersion Phase 1: v1.

2.4 Data minimum access request

Field	Ketentuan
ClientRequestId	ID dari client untuk idempotency. Retry tidak boleh membuat request kedua.
Requester	Diambil dari current/seeded user, bukan dipercaya dari payload bebas.
Application	CRM atau Finance Portal, terkait dengan System Owner.
Environment	NonProduction atau Production.
AccessLevel	Read atau Admin.
Justification	Wajib diisi.
Status	State model yang mencegah illegal transition.
PolicyVersion	v1 pada request yang dibuat di Phase 1.
Version / concurrency token	Untuk optimistic concurrency atau mekanisme setara.

2.5 Workflow Phase 1



High-risk apabila **Environment = Production** ATAU **AccessLevel = Admin**.

2.6 Business rules wajib - Phase 1

- Semua request membutuhkan Manager approval terlebih dahulu.
- Request non-high-risk menjadi Approved setelah Manager approve.
- Request high-risk berpindah ke System Owner approval setelah Manager approve.
- Manager hanya boleh memproses request direct report yang memang menunggu Manager approval.
- System Owner hanya boleh memproses request untuk aplikasi yang menjadi tanggung jawabnya.
- Requester tidak boleh approve request miliknya sendiri walaupun memiliki role approver.
- Reject wajib memiliki reason.
- Approved dan Rejected adalah terminal; transition berikutnya harus ditolak.
- Create request harus idempotent berdasarkan ClientRequestId. Duplicate concurrent request tidak boleh menghasilkan dua row bisnis.
- Approval/rejection harus terlindungi dari stale/concurrent action. Hanya satu transition terhadap version yang sama yang boleh berhasil.
- Setiap business-critical transition harus menyimpan audit event secara konsisten/atomic dengan perubahan state.
- PolicyVersion yang berlaku ketika request dibuat harus disimpan agar request lama tetap dapat dijelaskan ketika policy berubah di Phase 2.
- Authorization wajib server-side. Menyembunyikan tombol di UI bukan security boundary.

2.7 Required demo scenarios

Scenario	Expected outcome
Standard request	Alice -> CRM, NonProduction, Read. Bob approve -> Approved.
Production request	Alice -> CRM, Production, Read. Bob approve -> menunggu Carol; Carol approve -> Approved.
Admin access	Alice -> Finance Portal, NonProduction, Admin. Bob approve -> menunggu Dana.
Unauthorized approval	User yang bukan assigned approver memanggil endpoint -> ditolak backend.
Duplicate submit	ClientRequestId yang sama dikirim ulang / hampir bersamaan -> tetap satu request.
Concurrent action	Dua action terhadap version yang sama -> hanya satu berhasil; lainnya mendapat conflict yang jelas.
Rejected request	Reject reason tersimpan dan audit event tersedia; request tidak dapat diproses lagi.

3. Scope Teknis yang Wajib Dibangun

Your assignment
Rancang, implementasikan, test, review, dan dokumentasikan satu vertical slice end-to-end. Aplikasi tidak perlu luas; assessor harus dapat menjalankan critical flow create -> approve/reject -> audit secara konsisten.

3.1 Frontend

- User switcher/login simulasi menggunakan seeded users.
- Create access request form dengan validation feedback.
- My Requests: list sederhana dan detail request.
- Approval Inbox untuk Manager/System Owner.
- Request detail menampilkan current status dan audit timeline.
- Loading, empty, dan error/conflict state yang masuk akal untuk critical flow.

Tidak wajib: advanced search, pagination kompleks, design system, dashboard analytics, atau pixel-perfect UI.

3.2 Backend / API

- Endpoint/handler untuk create, list/detail, approval inbox, approve, dan reject.
- Server-side validation dan authorization.
- Idempotency untuk create request dengan perlindungan yang aman terhadap race.
- Optimistic concurrency atau mekanisme setara untuk approval/rejection.
- Error response yang membedakan validation, forbidden, not found, dan conflict.
- Basic logging yang cukup untuk menelusuri request/error. Full observability stack tidak diperlukan.

3.3 Database

- Relational schema dengan primary key, foreign key, unique constraint, dan constraint relevan.
- Migration/schema + seed dapat membangun database pada environment baru.
- Audit trail append-only dari normal application flow.
- Database constraint digunakan untuk invariant yang memang harus survive bug atau concurrency di application layer.

3.4 Automated tests

- Test business flow standard dan high-risk.
- Minimal satu authorization failure test.
- Minimal satu idempotency test.
- Minimal satu stale/concurrency behavior test.
- Semua test dapat dijalankan dengan command yang ditulis di README.

Frontend unit/component test bernilai tambah, tetapi tidak wajib bila critical business behavior sudah dibuktikan di backend/integration tests.

3.5 Sengaja tidak diminta

- OAuth/OIDC atau identity provider nyata.
- Email/notification service nyata.
- Microservices, message broker, event sourcing, Kubernetes, distributed cache, atau cloud deployment.
- Complex RBAC administration UI.
- Production-scale monitoring stack.

4. AI Usage, Integrity, dan Engineering Provenance

Prinsip integrity
Kami tidak menggunakan AI detector. AI boleh membantu seluruh proses. Yang diminta adalah evidence bahwa Anda mengarahkan tools, memverifikasi output, memahami risiko, dan bertanggung jawab atas submission. Orang lain tidak boleh mengerjakan assessment atas nama Anda.

4.1 Evidence yang wajib

Evidence	Isi minimum
Git history + tags	Gunakan Git sejak awal. Required tags: assessment-start dan phase-1-complete.
PLAN.md	Problem understanding, architecture/data model ringkas, implementation order, test strategy, 2-3 trade-off penting, serta perubahan plan selama implementasi.
AI_USAGE.md	Ringkas 2-5 interaksi AI yang paling berpengaruh: ask -> suggestion -> accepted/changed/rejected -> why. Tambahkan section "Three things AI got wrong".
REVIEW.md	Self-review production readiness: finding, severity, action, status, evidence. Sertakan known limitations dan deferred work.

Evidence	Isi minimum
INTEGRITY.md	Deklarasi ownership, tools AI utama, external human assistance, template/repository/snippet yang diadaptasi.
README.md	Setup, run, migrate/seed, test, demo users, dan demo flow.

4.2 Integrity declaration minimum

```
I confirm that this submission represents my own engineering work.
AI-assisted tools were used as documented in AI_USAGE.md.
I reviewed the submitted code and can explain its architecture, behavior,
known limitations, security implications, and trade-offs.

Date:
Primary AI tools used:
External human assistance: None / describe
Starter/template code used:
External repositories/snippets copied or adapted:
```

5. Git Workflow dan Phase Boundary

- Buat tag assessment-start sebelum implementasi utama.
- Buat PLAN.md awal sebelum coding besar dimulai.
- Setelah Phase 1 selesai dan test dijalankan, buat tag phase-1-complete.
- Jangan mengimplementasikan Phase 2 sebelum dokumen Change Request Phase 2 diberikan assessor.
- Jika submission berupa ZIP, pertahankan Git history (.git atau git bundle). Repository URL lebih disukai bila memungkinkan.

6. Reproducibility dan Submission Checklist

- Source frontend + backend dapat dijalankan mengikuti README.
- Database dapat dibuat ulang dari migration/schema + seed.
- Demo users dan demo flow tersedia.
- Automated tests berjalan dengan command terdokumentasi.
- Tag assessment-start dan phase-1-complete tersedia.
- PLAN.md, AI_USAGE.md, REVIEW.md, INTEGRITY.md, dan README.md tersedia.
- Tidak ada credential, token, atau data perusahaan nyata di repository.
- Jika timebox habis, state terakhir harus tetap runnable/stable dan deferred work dijelaskan.

Definition of done - Phase 1
Critical flow create -> approval/rejection -> audit dapat dijalankan. Business rules utama enforced di backend. Database reproducible. Test utama berjalan. Evidence engineering tersedia. Tag phase-1-complete tersedia.